

QuestBoard - Design Patterns Documentatie

Vak	C# Advanced
Type	ASP.NET Core Web API
Patterns	10
Framework	.NET 10 / ASP.NET Core

1. Projectoverzicht

QuestBoard is een gamified freelance platform waar taken worden gemodelleerd als quests. Freelancers verdienen XP, Gold en badges door quests te voltooien. Het project is gebouwd als ASP.NET Core Web API met een single-page frontend.

Structuur

```
src/QuestBoard.Api/
+-- Controllers/      API endpoints
+-- Models/Domain/    Quest, Freelancer, Client, Achievement
+-- Services/         Business logic
+-- Patterns/
| +-- Strategy/       Pricing + Matchmaking
| +-- Flyweight/      Skill/Badge pools
| +-- Decorator/      Quest modifiers
| +-- Bridge/         Notificaties
| +-- Interpreter/    Achievement DSL parser
| +-- Observer/       Event publisher + subscribers
| +-- Creational/     Singleton + Builder
| +-- Concurrency/    Producer-Consumer + Monitor
+-- Hubs/             SignalR real-time

tests/QuestBoard.Tests/
+-- Patterns/         Unit tests per pattern
```

Flow bij quest completion

```
POST /api/quests/{id}/complete
1. Decorator berekent finale XP/Gold (Urgent 1.5x, Featured 1.2x, Team Nx)
2. Event gaat op de queue (Producer-Consumer)
3. BackgroundService triggert Observer subscribers:
  - XpCalculatorSubscriber → XP + Gold + level-up
  - AchievementCheckerSubscriber → DSL regels (Interpreter) + badges (Flyweight)
  - LeaderboardUpdaterSubscriber → score bijwerken
  - SignalRBroadcasterSubscriber → live update naar browser
```

2. Singleton

Categorie: Creational | **Bestand:** Patterns/Creational/GameConfigurationManager.cs

Probleem: Configuratiewaarden (XP per difficulty, gold multipliers, team sizes) stonden verspreid over meerdere klassen.

Oplossing: Alle game-instellingen in 1 klasse via `Lazy<T>`, toegankelijk via `GameConfigurationManager.Instance`.

```
public sealed class GameConfigurationManager
{
    private static readonly Lazy<GameConfigurationManager> _instance =
        new(() => new GameConfigurationManager(),
        LazyThreadSafetyMode.ExecutionAndPublication);
    public static GameConfigurationManager Instance => _instance.Value;

    private GameConfigurationManager() { /* vult BaseXpByDifficulty dictionary */ }

    public int XpPerLevel { get; } = 1000;
    public decimal UrgentGoldMultiplier { get; } = 1.5m;
}
```

Voordelen: Alle configuratie op 1 plek. Thread-safe door `Lazy<T>`. Geen magic numbers.

Nadelen: Moeilijk te unit testen (geen DI). Globale state kan onverwachte side-effects geven.

Alternatieven: `appsettings.json` met `IOptions<T>` was testbaarder, maar voor game-constanten die nooit per environment veranderen voelt Singleton natuurlijker. Een static class mist lazy initialization.

3. Builder

Categorie: Creational | **Bestand:** Patterns/Creational/QuestBuilder.cs

Probleem: Een Quest heeft 14+ properties. Object initializers zijn onoverzichtelijk en je vergeet makkelijk verplichte velden.

Oplossing: Een fluent builder met validatie in `Build()` en defaults uit de Singleton.

```
var quest = QuestBuilder.Create()
    .WithTitle("Build REST API")
    .ForClient(client1.Id)
    .WithDifficulty(QuestDifficulty.Medium)
    .WithPricing(PricingType.Fixed, 500)
    .WithSkills("C#", "ASP.NET Core", "SQL")
    .AsUrgent(DateTime.UtcNow.AddDays(2))
    .Build(); // valideert verplichte velden + vult base XP uit Singleton
```

Voordelen: Leesbaar, ingebouwde validatie, automatische defaults.

Nadelen: Meer code dan een constructor. Overkill voor simpele objecten.

Alternatieven: Factory Method geeft minder controle. Telescoping constructor wordt onleesbaar bij 14+ velden.

4. Strategy

Categorie: Behavioral | **Bestanden:** Patterns/Strategy/ (7 bestanden)

Probleem: Meerdere manieren om prijs te berekenen (vast, veiling, dynamisch) en freelancers te matchen. If/else schaal niet.

Oplossing: Elk algoritme in een eigen klasse achter een interface, runtime wisselbaar.

IPricingStrategy	IMatchmakingStrategy
+-- FixedPricingStrategy	+-- SkillBasedMatchmaking
+-- AuctionPricingStrategy (20%)	+-- RatingBasedMatchmaking
+-- DynamicPricingStrategy	

```
// DynamicPricingStrategy past prijs aan op difficulty + urgentie
public decimal CalculatePrice(decimal basePrice, int difficulty, DateTime? deadline)
{
    var multiplier = DifficultyMultipliers.GetValueOrDefault(difficulty, 1.0m);
    var price = basePrice * multiplier;
    if (deadline.HasValue && (deadline.Value - DateTime.UtcNow).TotalDays <= 3)
        price *= 1.25m; // urgentietoeslag
    return price;
}
```

Voordelen: Open/Closed Principle. Makkelijk te testen per strategie. Runtime wisselbaar.

Nadelen: Meer klassen dan if/else. Client moet weten welke strategieën er zijn.

Alternatieven: Switch/if-else vereist aanpassing bij elke nieuwe strategie. Command pattern is meer voor acties, niet voor verwisselbare algoritmes.

5. Decorator

Categorie: Structural | **Bestanden:** Patterns/Decorator/ (6 bestanden)

Probleem: Quests kunnen modifiers hebben (Urgent, Featured, Team) die combineerbaar zijn. Overerving geeft class-explosie.

Oplossing: Decorators die om elkaar heen gewrapped worden, elk past 1 modifier toe.

IQuestBehavior
+-- BaseQuestBehavior
+-- QuestDecorator (abstract)
+-- UrgentQuestDecorator (1.5x gold, 2x XP)
+-- FeaturedQuestDecorator (1.2x gold, +50 XP)
+-- TeamQuestDecorator (gold x teamSize, XP x 0.8)

```
IQuestBehavior behavior = new BaseQuestBehavior();
if (quest.IsUrgent) behavior = new UrgentQuestDecorator(bbehavior);
if (quest.IsFeatured) behavior = new FeaturedQuestDecorator(bbehavior);
if (quest.IsTeamQuest) behavior = new TeamQuestDecorator(bbehavior,
```

```
quest.MaxTeamSize);  
// Urgent + Featured met baseGold 500 → 500 * 1.5 * 1.2 = 900
```

Voordelen: Vrij combineerbaar zonder class-explosie. Elke decorator heeft 1 verantwoordelijkheid.

Nadelen: Volgorde van wrapping beïnvloedt resultaat. Debugging door meerdere lagen.

Alternatieven: Overerving geeft bij 3 modifiers al 7 subklassen. Decorator houdt het op 3 klassen.

6. Observer

Categorie: Behavioral | **Bestanden:** Patterns/Observer/ (8 bestanden)

Probleem: Na quest completion moeten 4 dingen gebeuren (XP, achievements, leaderboard, SignalR). Alles in 1 methode geeft tight coupling.

Oplossing: Een publisher die events broadcast naar geregistreerde subscribers.

```
QuestEventPublisher  
+-- XpCalculatorSubscriber      (XP, Gold, level-up)  
+-- AchievementCheckerSubscriber (DSL regels + badges)  
+-- LeaderboardUpdaterSubscriber (score bijwerken)  
+-- SignalRBroadcasterSubscriber (real-time naar browser)
```

```
public void Notify(QuestCompletedEvent evt)  
{  
    foreach (var subscriber in _subscribers)  
        subscriber.OnQuestCompleted(evt);  
}
```

Voordelen: Losse koppeling. Nieuwe functionaliteit = nieuwe subscriber registreren.

Nadelen: Volgorde van notificatie niet gegarandeerd. Debugging bij veel subscribers lastig.

Alternatieven: Mediator is meer voor bidirectionele communicatie. Observer past beter voor 1-naar-veel broadcast.

7. Interpreter

Categorie: Behavioral | **Bestanden:** Patterns/Interpreter/ (6 bestanden)

Probleem: Achievement-regels moeten configureerbaar zijn zonder code te wijzigen.

Oplossing: Een DSL waarmee regels als strings worden opgeslagen en geparsed naar een expression tree.

```
"quests.completed >= 25 and rating.avg >= 4.0"  
→ AndExpression(  
    ComparisonExpression("quests.completed", ">=", 25),  
    ComparisonExpression("rating.avg", ">=", 4.0)  
)
```

```
// ComparisonExpression evalueert een property tegen een waarde
public bool Interpret(AchievementContext context)
{
    var actual = context.GetValue(_propertyName);
    return _op switch
    {
        ">=" => actual >= _value,
        ">"  => actual > _value,
        "<=" => actual <= _value,
        "==" => Math.Abs(actual - _value) < 0.0001,
        _    => throw new InvalidOperationException($"Unknown operator: {_op}")
    };
}
```

Voordelen: Regels configureerbaar zonder code te wijzigen. Nieuwe achievement = string toevoegen.

Nadelen: Beperkte DSL (alleen vergelijkingen met AND/OR). String-parsing is foutgevoelig.

Alternatieven: Hardcoded if/else per achievement schaaft niet. Specification pattern past minder bij het parsen van een eigen taal.

8. Bridge

Categorie: Structural | **Bestanden:** Patterns/Bridge/ (7 bestanden)

Probleem: 2 notificatietypes × 3 kanalen = 6 combinaties. Zonder Bridge een klasse per combinatie.

Oplossing: Abstractie (notificatietype) en implementatie (kanaal) gescheiden.

Notification (abstract)	INotificationSender (interface)
+-- QuestAlert	+-- EmailSender
+-- AchievementAlert	+-- PushNotificationSender
	+-- WebhookSender
→ 2 types × 3 kanalen = 6 combinaties met slechts 5 klassen	

```
public class QuestAlert : Notification
{
    public override string Notify(string recipient)
        => _sender.Send(recipient, $"Quest Update: {QuestTitle}", $"Status: {QuestStatus}");
}
```

Voordelen: Twee dimensies onafhankelijk uitbreidbaar. Elke combinatie werkt automatisch.

Nadelen: Extra abstractielaag. Kan overkill voelen bij weinig types/kanalen.

Alternatieven: Zonder Bridge: aparte klassen per combinatie (QuestAlertEmail , QuestAlertPush , etc.).

9. Flyweight

Categorie: Structural | **Bestanden:** Patterns/Flyweight/ (4 bestanden)

Probleem: Veel freelancers delen dezelfde skills en badges. Zonder sharing wordt elk object apart aangemaakt.

Oplossing: SkillFactory en BadgeFactory cachen immutable objecten in een ConcurrentDictionary. Twee keer GetSkill("C#") geeft exact hetzelfde object.

```
public SkillFlyweight GetSkill(string name, string category = "General", string
iconUrl = "")
{
    return _pool.GetOrAdd(name, key => new SkillFlyweight(key, category, iconUrl));
}
```

Voordelen: Geheugen besparen. Thread-safe. Immutable objecten = geen onbedoelde wijzigingen.

Nadelen: Alleen nuttig bij veel duplicaten. Objecten moeten echt immutable zijn.

Alternatieven: Strings voor skills mist metadata. Prototype pattern maakt kopieën; Flyweight deelt exact dezelfde objecten.

10. Monitor

Categorie: Concurrency | **Bestand:** Patterns/Concurrency/QuestAcceptanceLock.cs

Probleem: Twee gebruikers klikken tegelijk op "Accept" → race condition.

Oplossing: Per quest een SemaphoreSlim(1,1) als mutex. Fine-grained: andere quests worden niet geblokkeerd.

```
public T ExecuteWithLock<T>(Guid questId, Func<T> action)
{
    var semaphore = _locks.GetOrAdd(questId, _ => new SemaphoreSlim(1, 1));
    semaphore.Wait();
    try { return action(); }
    finally { semaphore.Release(); }
}

// Gebruik:
_acceptanceLock.ExecuteWithLock(questId, () => {
    if (quest.Status != QuestStatus.Open) return null;
    quest.AssignedFreelancerId = freelancerId;
    quest.Status = QuestStatus.InProgress;
    return MapToResponse(quest);
});
```

Voordelen: Voorkomt race conditions. Fine-grained locking per quest.

Nadelen: Extra overhead per quest. Dictionary groeit onbeperkt (opgelost met TryRemoveLock()).

Alternatieven: Globale lock(obj) blokkeert alle quests. Mutex is zwaarder (cross-process). SemaphoreSlim is lichter en perfect voor in-process locking.

11. Producer-Consumer

Categorie: Concurrency | **Bestanden:** Patterns/Concurrency/ (3 bestanden)

Probleem: Observer subscribers synchroon op de HTTP thread draaien maakt de API traag.

Oplossing: Event op een bounded `Channel<T>` queue. HTTP retournt meteen. `BackgroundService` verwerkt async.

```
// Producer (in QuestService.CompleteQuest)
await _eventQueue.EnqueueAsync(new QuestCompletedEvent(quest, freelancer, gold,
xp));

// Consumer (BackgroundService)
await foreach (var evt in _eventQueue.DequeueAllAsync(stoppingToken))
    _publisher.Notify(evt); // triggert alle Observer subscribers
```

Voordelen: HTTP thread niet geblokkeerd. Gegarandeerde verwerking (bounded channel).

Nadelen: Events verloren bij crash (geen persistentie). Async debugging lastiger.

Alternatieven: `Task.Run()` fire-and-forget verliest controle. Message broker (RabbitMQ) te zwaar. `Channel<T>` is de .NET-native oplossing.

12. SignalR Real-Time

Bestanden: Hubs/QuestBoardHub.cs , Patterns/Observer/SignalRBroadcasterSubscriber.cs

Observer subscriber die events via WebSocket naar de browser broadcast. De frontend toont direct een melding en refresht automatisch.

```
// SignalRBroadcasterSubscriber stuurt naar alle connected clients
_ = _hubContext.Clients.All.QuestCompleted(new {
    QuestTitle = evt.Quest.Title, FreelancerName = evt.Freelancer.Name,
    GoldEarned = evt.GoldEarned, XpEarned = evt.XpEarned
});
```

13. Hoe de patterns samenwerken

Het meest complexe punt is `AchievementCheckerSubscriber` , waar 4 patterns samenkomen:

```
// Observer: gettriggerd na quest completion
public void OnQuestCompleted(QuestCompletedEvent evt)
{
    foreach (var achievement in _dataStore.Achievements.Values)
    {
        var expression = AchievementRuleParser.Parse(achievement.DslRule); //
Interpreter
        if (expression.Interpret(context) &&
            !freelancer.Badges.Contains(achievement.BadgeName))
```

```

        {
            var badge = _badgeFactory.GetBadge(achievement.BadgeName, ...); //
Flyweight
            new AchievementAlert(new EmailSender(), ...).Notify(freelancer.Email);
// Bridge
        }
    }
}

```

Volledige flow bij quest completion:

1. **Builder + Singleton** → Quest aangemaakt met defaults
2. **Strategy** → Prijs berekend met gekozen algoritme
3. **Monitor** → Quest veilig geaccepteerd (per-quest lock)
4. **Decorator** → Finale XP/Gold met gestapelde modifiers
5. **Producer-Consumer** → Event op queue, HTTP retournt direct
6. **Observer** → Publisher broadcast naar 4 subscribers
7. **Interpreter** → Achievement-regels geparsed en geëvalueerd
8. **Flyweight** → Badges uit gedeelde pool
9. **Bridge** → Notificatie via juiste kanaal
10. **SignalR** → Live update naar browser